



Trabajo de Investigación: Expo (React Native)

Justin Gabriel Moreira Matarrita

Abigail Ramírez Chavarría

Luis Fernando Rosales Vargas

Luis Alejandro Salazar Vargas

Fabian Yosiney Sanchez Salinas

Facultad de Ciencias Exactas y Naturales

EIF 411: Diseño y Programación de Plataformas Móviles

Mag. Rachel Bolívar Morales

6 de junio de 2026

1. Historia y origen del framework

Para entender el origen de Expo es necesario analizar el nacimiento de React Native. En 2015 Facebook liberó esta tecnología como código abierto con la promesa de que los desarrolladores web construirían aplicaciones móviles nativas utilizando los mismos patrones de React (React Native Team, 2026a). Aunque React Native se representó como un avance importante su ecosistema inicial presentaba una barrera significativa porque configurar el entorno para iOS y Android requería instalar Xcode Android Studio SDKs específicos y dependencias complejas. Para desarrolladores web o equipos pequeños esa curva de entrada resultaba disuasiva.

Expo nació como respuesta a esa fricción. La empresa Expo Inc lanzó las primeras versiones en 2016 pero ganó terreno en 2017 con sus primeras versiones estables (Expo, s.f.). La propuesta inicial eliminaba la configuración de los entornos nativos locales permitiendo ejecutar una aplicación en el dispositivo físico en minutos escaneando un código QR mediante la aplicación Expo Go.

El framework evolucionó de forma significativa, debido a las limitaciones iniciales para poder acceder a funcionalidades nativas. Si una librería de terceros requería módulos nativos adicionales, el proyecto quedaba fuera del flujo de trabajo. Por ello el equipo introdujo en 2020 el *Bare Workflow* un modo para acceder directamente al código nativo de Android e iOS sin perder las APIs del SDK de Expo (Expo, s.f.).

Un hito relevante fue la introducción de Expo Application Services (EAS) a finales de 2020, un conjunto de servicios en la nube para compilar, firmar y distribuir aplicaciones sin necesidad de un entorno local. Con EAS Build los equipos generan archivos binarios en servidores remotos, simplificando considerablemente el proceso de despliegue (Vatne, 2020). Entre 2023 y 2024, Expo adoptó Expo Router, un sistema de enrutamiento basado en archivos que incorpora conceptos modernos de enrutamiento provenientes del desarrollo web,

donde la ubicación de un archivo dentro del directorio `app/` define automáticamente su ruta (Expo Team, 2026). Actualmente el framework refleja una base sólida con actualizaciones frecuentes, que soportan las últimas versiones de React Native, habiendo publicado el SDK 50 en 2024 (Vatne, 2024).

2. Arquitectura y funcionamiento

Para comprender cómo funciona Expo, es necesario partir de la arquitectura de React Native, sobre el cual Expo se basa. React Native opera bajo un modelo donde el código JavaScript se ejecuta en un motor separado del hilo principal del sistema operativo (React Native Team, 2026a). Históricamente, esta comunicación entre el mundo JavaScript y el mundo nativo se realizaba a través de un puente asíncrono “JS Bridge”, que serializaba los mensajes en formato JSON y los enviaba en ambas direcciones. Esta arquitectura tenía implicaciones de rendimiento, especialmente en escenarios de animaciones complejas o manipulación intensiva de la UI.

A partir de React Native 0.68 el proyecto comenzó la transición hacia la nueva arquitectura basada en JavaScript Interface (JSI), la cual reemplaza el puente asíncrono por una interfaz directa en C++ para invocar funciones nativas de forma sincrónica (React Native Team, 2026b). Junto con JSI también aparecieron Fabric como un nuevo motor de renderizado que opera directamente en el hilo principal y TurboModules como sistema de módulos nativos de carga diferida que elimina la inicialización anticipada al arrancar la aplicación. Expo incorporó soporte para esta arquitectura progresivamente y la habilitación de JSI es un estándar en sus versiones recientes (Expo, s.f.).

El flujo de ejecución inicia cuando un desarrollador escribe un componente JSX, ese código pasa por el compilador de Babel, que lo transforma en JavaScript plano compatible con el motor Hermes. Hermes es un motor de código abierto optimizado para dispositivos

móviles que genera bytecode antes de la ejecución, reduciendo el tiempo de arranque respecto al antiguo JavaScriptCore (Horowitz, 2019).

Expo agrega sobre esta base el Expo SDK, que expone funcionalidades nativas del sistema operativo como la cámara GPS, sistema de archivos, notificaciones y sensores mediante una API unificada que abstrae con las diferencias entre iOS y Android. Los módulos están implementados en Kotlin para Android y Swift para iOS, conectándose al entorno JavaScript con el sistema de módulos nativos (Expo, s.f.). En el *Managed Workflow* la aplicación incluye un conjunto por defecto en estos módulos, mientras que en el *Bare Workflow* el desarrollador controla qué módulos incluir, pudiendo extenderlos con cualquier librería nativa compatible.

3. Lenguajes utilizados

El lenguaje principal en Expo es JavaScript, específicamente TypeScript establecido como el estándar en proyectos profesionales. Meta eligió JavaScript para poder aprovechar la gran cantidad de desarrolladores web con experiencia en este lenguaje, permitiéndoles construir aplicaciones móviles sin aprender un nuevo lenguaje. Las nuevas plantillas del framework se generan con TypeScript por defecto y la documentación oficial prioriza este lenguaje como primera opción (Wei y Gerleman 2023).

JSX es la extensión sintáctica que permite escribir estructuras de interfaz de usuario mediante un modelo declarativo, donde los componentes describen la interfaz según su estado. En Expo, los componentes se organizan en árboles jerárquicos donde los datos fluyen mediante props y el estado se gestiona con hooks como useState o librerías de manejo de estado global como Zustand y Redux Toolkit (React Native Team, 2026a).

La capa nativa de los módulos de Expo está escrita en Kotlin para Android y Swift para iOS, garantizando un acceso optimizado a las APIs del sistema operativo. Esta separación entre JavaScript para la lógica y Kotlin o Swift para el hardware, permite generar

interfaces nativas reales, superando los enfoques basados en WebView que degradan la experiencia del usuario (Expo, s.f.). En el Managed Workflow este proceso es transparente para el desarrollador pero resulta relevante al depurar el rendimiento o extender el SDK con módulos propios.

4. Casos reales de uso

El uso de React Native y Expo en entornos de producción de alto tráfico constituye uno de los argumentos técnicos más sólidos a favor del ecosistema. El caso más documentado es el de Shopify, que en 2020 anunció formalmente la adopción de React Native para el desarrollo de nuevas aplicaciones móviles. La decisión respondió a la necesidad de unificar los equipos de iOS y Android bajo una base de código compartida, eliminando la duplicación de lógica de negocio y acelerando los ciclos de entrega sin sacrificar la experiencia nativa para sus comerciantes (Thawar, 2020). Este caso es relevante no solo por el tamaño de la empresa, sino porque Shopify documentó el proceso de migración de forma pública, incluyendo los retos encontrados y los cambios organizacionales que implicó.

Discord representa otro ejemplo consolidado dentro de la industria. La empresa utiliza React Native en su aplicación móvil aprovechando la posibilidad de compartir lógica de negocio con su cliente web, ambos desarrollados sobre la base de React. Esta decisión redujo la fragmentación entre plataformas y permitió que un equipo móvil relativamente pequeño mantuviera paridad de funcionalidades con el cliente de escritorio (Rozelle, 2018). Microsoft también ha adoptado React Native en proyectos específicos bajo su iniciativa react-native-windows, extendiendo el ecosistema fuera de iOS y Android hacia Windows, lo que evidencia la confianza empresarial que el ecosistema ha ganado fuera del contexto estrictamente móvil (Microsoft, s.f.).

En proyectos de menor escala, Expo cobra especial relevancia para startups y equipos que operan bajo restricciones de tiempo y presupuesto. La posibilidad de mantener un único

repositorio que compile para ambas plataformas representa una ventaja económica directa, ya que elimina la necesidad de contratar equipos especializados separados para iOS y Android (Thawar, 2020). Para una problemática como la de sistemas de reservas y eventos, donde el ciclo de iteración y la velocidad de lanzamiento son más críticos que el acceso a hardware especializado, este patrón de adopción tiene una correspondencia directa con el tipo de proyecto que el grupo debe desarrollar.

5. Comparación técnica con Android nativo

La comparación entre Expo y el desarrollo nativo para Android involucra dimensiones técnicas, organizacionales y económicas que dependen directamente del tipo de proyecto. En términos de rendimiento, una aplicación construida en Kotlin con la API nativa de Android tiene acceso directo al hilo de renderizado del sistema sin intermediarios, traducándose en tiempos de respuesta cortos con mayor consistencia en las animaciones intensivas (React Native Team, 2026a). Una aplicación desarrollada con Expo introduce cierta latencia extra derivada de la comunicación entre el motor de JavaScript y el sistema nativo. Esta diferencia es imperceptible en la mayoría de los casos de uso como listas, formularios, navegación y consultas a APIs, pero puede manifestarse en aplicaciones que requieren procesamiento gráfico, realidad aumentada o acceso constante a sensores de alta frecuencia.

Respecto al acceso a hardware, con Android nativo el desarrollador tiene acceso completo y actualizado a todas las APIs que Google publica, incluyendo nuevas funcionalidades de Bluetooth, NFC, biometría o funciones específicas de chips Qualcomm o MediaTek. Con Expo en modo gestionado, el desarrollador depende de que el SDK incluya soporte para esa funcionalidad, mientras que en el *Bare Workflow* puede instalar la librería nativa correspondiente de React Native, aunque eso requiere configurar el proyecto nativo

manualmente (Expo, s.f.). Esta limitación se ha reducido con el tiempo pero sigue siendo un factor a considerar para aplicaciones con requerimientos de hardware muy específicos.

Desde la perspectiva del desarrollo, configurar un proyecto nuevo con Android nativo en Kotlin implica instalar Android Studio, configurar un emulador o dispositivo físico y familiarizarse con la estructura de proyectos Gradle, los ciclos de build y el sistema de manifiestos. Con Expo, el proceso se reduce a ejecutar `npx create-expo-app` en la terminal y escanear un código QR, logrando la primera ejecución en minutos frente a las horas que puede tomar el entorno nativo (Expo, s.f.). Esto impacta directamente en la productividad inicial del equipo y en la curva de aprendizaje de desarrolladores que provienen del entorno web.

En cuanto al tamaño de la aplicación compilada, con Expo suele tener un tamaño inicial mayor que las aplicaciones nativas, debido a que el bundle incluye el runtime de JavaScript, el motor Hermes, los módulos del SDK precompilados y las dependencias de React Native (Expo, s.f.; Horowitz, 2019). Este aspecto es relevante en mercados donde el tamaño de descarga influye en las tasas de instalación especialmente en regiones con conectividad limitada o planes de datos restrictivos.

Finalmente, Android nativo cuenta con el respaldo directo de Google, documentación oficial actualizada y una comunidad con experiencia acumulada. React Native y Expo poseen una comunidad muy activa en GitHub, Discord y Stack Overflow, con actualizaciones frecuentes y una cultura de código abierto consolidada (React Native Team, 2026b). Sin embargo, la velocidad a la que el ecosistema evoluciona puede representar deuda técnica si las dependencias no se actualizan con regularidad, ya que los cambios de la arquitectura no siempre son compatibles con versiones anteriores.

6. Ventajas y desventajas

6.1 Ventajas

La ventaja más tangible de Expo es la reducción del tiempo de configuración, lo que disminuye la curva de aprendizaje inicial para equipos con experiencia en JavaScript o React. En entornos académicos o de startups, donde los plazos son cortos y escasean los especialistas en cada plataforma, iniciar el desarrollo sin configurar entornos nativos complejos tiene un valor práctico significativo (Expo, s.f.). Además, la función de *hot reload*, que actualiza la interfaz en tiempo real mientras se edita el código, acelera el ciclo de iteración de forma considerable al eliminar las recompilaciones completas.

Compartir código entre iOS y Android constituye otra ventaja estructural. En proyectos con lógica de negocio en común, Expo permite mantener un único repositorio, reducir errores por implementaciones paralelas y concentra el esfuerzo de mantenimiento. El ecosistema facilita la adopción de buenas prácticas mediante la integración nativa con TypeScript, herramientas de linting, Jest para pruebas automatizadas y servicios de CI/CD (Wei y Gerleman, 2023). Asimismo, EAS Build y EAS Submit simplifican el despliegue en las tiendas de aplicaciones, eliminando la gestión local de certificados de firma (Vatne, 2020).

Por último, Expo Router representa un avance significativo en la organización del código. Al adoptar un sistema de enrutamiento basado en archivos, donde la ubicación dentro del directorio *app/* determina la ruta, se simplifica la configuración de navegación y se reduce el código repetitivo, facilitando una estructura de proyecto más ordenada y mantenible (Expo Team, 2026).

6.2 Desventajas

La principal desventaja de Expo, específicamente en el Managed Workflow, es la pérdida de control sobre la capa nativa. Cuando una librería de terceros requiere modificaciones en el archivo *AndroidManifest.xml*, en el Podfile de iOS o en configuraciones

nativas específicas, el proyecto se debe migrar al Bare Workflow. Esto implica un costo de configuración si el equipo no cuenta con experiencia en herramientas nativas, y puede producir inconsistencias en el entorno si no se gestiona con cuidado (Expo, s.f.).

El tamaño del ejecutable final es otra limitación relevante. A diferencia de una aplicación nativa eficiente, el paquete de JavaScript puede crecer de forma descontrolada en proyectos grandes si no se aplican técnicas de *code splitting* y *tree shaking*. Hermes contribuye a mejorar el rendimiento y optimizar la ejecución de aplicaciones React Native en dispositivos móviles (Horowitz, 2019). Adicionalmente, la depuración de errores nativos es más compleja en Expo, ya que el rastreo de fallos en el código base nativo no siempre se traduce de forma legible al contexto de JavaScript.

Finalmente, la dependencia de Expo para las actualizaciones del SDK representa un riesgo a largo plazo. Cuando React Native lanza una nueva versión, los usuarios de Expo deben esperar a que el equipo de la plataforma la integre en su SDK, lo que introduce un retraso de semanas o inclusive meses (Vatne, 2024). En proyectos donde el acceso inmediato a las últimas APIs del sistema operativo es crítico, esta latencia puede resultar una gran problemática.

7. Retos encontrados

La documentación de la comunidad se registra de forma recurrente varios retos que los equipos enfrentan durante sus primeros proyectos con esta tecnología. El primero es la confusión entre los modos de trabajo *Managed* y *Bare Workflow* y implicaciones a medida que el proyecto crece. Muchos equipos inician en el modo gestionado por su simplicidad, pero descubren a mitad del desarrollo que necesitan una funcionalidad nativa no cubierta por el SDK, lo que los obliga a realizar una migración compleja en un momento poco conveniente (Expo, s.f.).

La gestión de dependencias es otro punto documentado ampliamente. El ecosistema de React Native destaca por su fragmentación, ya que las librerías de terceros mantienen ritmos de actualización diferentes, provocando varios conflictos de versiones difíciles de detectar. El comando *npx expo install*, que instala las versiones explícitamente compatibles con el SDK en uso, mitiga este problema, pero no lo elimina por completo en proyectos que combinan múltiples dependencias externas (Expo, s.f.).

El rendimiento en listas de gran volumen representa un reto técnico crítico. El componente *FlatList*, el estándar para renderizar listas largas, puede presentar comportamientos subóptimos en dispositivos de gama baja si no se configura con propiedades como *windowSize*, *maxToRenderPerBatch* y *getItemLayout*. Los desarrolladores web suelen asumir que la virtualización es automática y suficiente, encontrando problemas de rendimiento al probar en dispositivos reales de gama baja (React Native Team, 2026a).

La configuración de las notificaciones push, un requerimiento técnico obligatorio de este proyecto, presenta su propia complejidad. Aunque *Expo Notifications* simplifica el proceso respecto a la integración directa con Firebase Cloud Messaging para Android y APNs para iOS, exige gestionar tokens, manejar permisos explícitos en cada plataforma y comprender el comportamiento en primer plano frente al segundo plano (Expo, s.f.). Los equipos sin experiencia previa suelen subestimar el tiempo necesario para implementar esta lógica de forma adecuada.

Por último, el proceso de compilación y publicación con EAS, aunque es más simple que la alternativa local, posee particularidades importantes. Los tiempos de espera en los servidores de EAS pueden ser extensos en cuentas gratuitas, y la configuración del archivo *eas.json* requiere entender los perfiles de compilación, las credenciales de firma y las diferencias entre entornos de desarrollo, preview y producción (Expo, 2026). Para equipos

nuevos en el ciclo de vida móvil, esta fase puede consumir un tiempo desproporcionado en las etapas finales del proyecto.

8. Análisis crítico

La decisión entre Managed Workflow y Bare Workflow es, probablemente, la más importante y la que menos atención recibe en los recursos introductorios. Para una aplicación de reservas y eventos, donde las notificaciones push y un componente de calendario son requisitos centrales, esa elección define desde el inicio qué librerías son viables y cuáles no. Empezar en modo gestionado por comodidad y descubrir a mitad del desarrollo que una dependencia necesaria requiere configuración nativa es el escenario que el grupo considera más probable y más costoso de corregir. La recomendación del equipo es evaluar desde la fase de diseño si todas las dependencias previstas son compatibles con el Managed Workflow, y tomar la decisión de migrar al Bare Workflow antes de que el proyecto tenga código de producción, no para después.

Las notificaciones push son el otro componente que genera más dudas. La abstracción que ofrece Expo Notifications funciona bien en casos simples, pero el manejo de permisos por plataforma y el comportamiento en segundo plano tienen suficiente variación entre iOS y Android como para que la implementación tome más tiempo del que aparenta. Para una problemática donde las notificaciones no son un extra sino parte del flujo principal para confirmar una reserva, alertar sobre cambios de horario, recordar un evento próximo, ese tiempo importa y debe considerarse en la planificación del trabajo.

En términos generales, el grupo considera que Expo es una elección adecuada para este proyecto. Los requerimientos no exigen acceso a hardware especializado ni procesamiento gráfico intensivo, y la posibilidad de compartir código entre plataformas es una ventaja real para un equipo de cinco personas con tiempo limitado. La duda no es si el

framework sirve para esto, sino si el grupo va a tomar las decisiones arquitectónicas correctas antes de necesitarlas.

Sin embargo, afirmar que Expo es la herramienta correcta para cualquier escenario. Si la problemática asignada hubiera implicado integración con hardware específico, lectores de huella dactilar de fabricantes concretos, NFC para control de acceso físico, o procesamiento de video en tiempo real, la elección habría sido diferente. En esos casos, el costo de mantener módulos nativos propios dentro de un proyecto Expo o de depender de que el SDK cubra esa funcionalidad específica superaría con creces las ventajas de la configuración simplificada. Para aplicaciones con requerimientos de hardware no convencionales o con necesidades de rendimiento en animaciones complejas, Android nativo con Kotlin sigue siendo la opción más predecible y controlable. La pertinencia de Expo es real, pero está acotada a un perfil de proyecto específico, y reconocer ese límite es parte del criterio técnico que el grupo considera necesario para tomar decisiones de arquitectura responsables.

Bibliografía

Expo. (2026). *EAS Build documentation*. <https://docs.expo.dev/build/introduction>

Expo. (s.f.). *Expo documentation*. Recuperado el 26 de mayo de 2026, de <https://docs.expo.dev>

Expo Team. (2026). *Introduction to Expo Router*. Expo Documentation. <https://docs.expo.dev/router/introduction/>

Horowitz, M. (2019). *Hermes: An open source JavaScript engine optimized for mobile apps, starting with React Native*. Engineering at Meta. <https://engineering.fb.com/2019/07/12/android/hermes>

Microsoft. (s.f.). *React Native for Windows + macOS*. Microsoft Open Source. <https://microsoft.github.io/react-native-windows/>

React Native Team. (2026a). *React Native documentation*. React Native.
<https://reactnative.dev/docs/getting-started>

React Native Team. (2026b). *The new architecture overview*. React Native.
<https://reactnative.dev/docs/the-new-architecture/landing-page>

Rozelle, C. (2018). How Discord achieves native iOS performance with React Native.
 Engineering at Discord.
<https://discord.com/blog/how-discord-achieves-native-ios-performance-with-react-native>

Thawar, F. (2020). *React Native is the future of mobile at Shopify*. Shopify Engineering.
<https://shopify.engineering/react-native-future-mobile-shopify>

Vatne, B. (2020). *Expo Application Services (EAS): Build and Submit*. Expo Blog.
<https://blog.expo.dev/expo-application-services-eas-build-and-submit-fc1d1476aa2e>

Vatne, B. (2024). *Expo SDK 50*. Expo. <https://expo.dev/changelog/2024/01-18-sdk-50>

Wei, L., & Gerleman, N. (2023). *First-class Support for TypeScript*. React Native.
<https://reactnative.dev/blog/2023/01/03/typescript-first#thank-you-to-the-react-native-typescript-community>